

1. Introdução

Realizado por Pedro d'Angelis Otero e Davi Luigi Araujo Amaral, este trabalho final tem como objetivo a modelagem e implementação de três máquinas abstratas fundamentais da Teoria da Computação: o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) e a Máquina de Turing (MT). Utilizando a linguagem C#, o trabalho busca traduzir definições matemáticas rigorosas em simuladores funcionais capazes de processar linguagens regulares, livres de contexto e sensíveis ao contexto.

A relevância do estudo dessas máquinas transcende a teoria pura. Conforme demonstrado na literatura, os AFDs podem ser aplicados desde a predição de comportamentos baseada na hierarquia de necessidades de Maslow até sistemas embarcados para validação de eventos esportivos. Além disso, a implementação de uma Máquina de Turing para a função $f(n) = n + 1$ reforça o legado de Alan Turing, cujo trabalho estabeleceu os fundamentos da computação moderna e da Inteligência Artificial.

2. Fundamentação Teórica

A base teórica deste trabalho sustenta-se nas definições formais que descrevem o poder e as limitações de cada modelo computacional citado, conforme consolidado por autores como Hopcroft e Sipser.

2.1 Autômato Finito Determinístico (AFD)

O Autômato Finito Determinístico é representado pela 5-tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

onde:

- Q representa o conjunto de estados;
- Σ representa o alfabeto de entrada;
- δ representa a função de transição;

- q_0 representa o estado inicial;
- F representa o conjunto de estados finais.

Trata-se de um reconhecedor de linguagens regulares em que cada par formado por estado e símbolo possui exatamente um próximo estado associado.

Exemplo Manual (L1 – Cadeias terminadas em "ab")

Para a entrada "bab", a computação ocorre da seguinte forma:

$$\delta(q_0, b) = q_0$$

$$\delta(q_0, a) = q_1$$

$$\delta(q_1, b) = q_2$$

Como q_2 pertence ao conjunto F , a cadeia é aceita.

2.2 Autômato de Pilha (AP)

O Autômato de Pilha é definido pela 7-tupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

onde Γ representa o alfabeto da pilha e Z_0 o símbolo inicial da pilha.

Esse modelo utiliza uma estrutura de memória do tipo LIFO (Last In, First Out), permitindo o reconhecimento de linguagens livres de contexto.

Exemplo Manual (L2 – Linguagem $a^n b^n$)

Na leitura da cadeia "aabb":

1. Os símbolos "a" são empilhados.
2. Ao iniciar a leitura dos símbolos "b", ocorre o desempilhamento.
3. Ao final da entrada, se a pilha estiver vazia, a cadeia é aceita.

2.3 Máquina de Turing (MT)

A Máquina de Turing é definida pela 7-tupla:

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

Diferentemente dos autômatos anteriores, a MT possui uma fita potencialmente infinita, permitindo operações de leitura, escrita e movimentação do cabeçote para ambos os lados.

Exemplo Manual (Função $f(n) = n + 1$)

Considere a entrada unária:

111

O cabeçote percorre todos os símbolos "1" até encontrar o primeiro branco ($_$). Em seguida, substitui esse branco por "1" e entra no estado de aceitação.

Saída produzida:

1111

3. Descrição da Implementação

A transposição dos modelos matemáticos para código C# seguiu uma abordagem de alta fidelidade às definições formais, evitando o uso de bibliotecas externas específicas para autômatos.

3.1 Autômato Finito Determinístico (AFD)

O AFD foi implementado utilizando a estrutura HashSet para representar os conjuntos Q e F, garantindo unicidade dos elementos e busca eficiente em tempo constante. Já a função de transição δ foi modelada utilizando a coleção:

Dictionary<(string, char), string>

Permitindo uma tradução direta da relação de (Estado Atual $\times$ Símbolo) $\rightarrow$ Próximo Estado

O sistema também suporta carregamento dinâmico através de arquivos JSON, possibilitando a simulação de diferentes autômatos sem alterações no código-fonte.

3.2 Autômato de Pilha (AP)

O Autômato de Pilha (AP) utiliza a estrutura nativa Stack para representar a pilha.

Para o reconhecimento de palíndromos (L3), foi implementada uma estratégia de backtracking baseada em recursão, permitindo explorar caminhos alternativos da computação quando necessário.

Durante a execução, o simulador exibe continuamente:

- Estado atual;
- Conteúdo da pilha;
- Parte restante da cadeia de entrada.

Essa visualização permite acompanhar a evolução da computação passo a passo.

3.3 Máquina de Turing (MT)

A implementação da Máquina de Turing foi desenvolvida com foco em robustez e segurança da simulação.

Fita Dinâmica

A fita é representada pela estrutura:

Dictionary<int, char>

na qual cada posição da fita corresponde a uma chave do dicionário. Essa abordagem elimina limitações de tamanho e permite expansão infinita em ambas as direções.

Função de Transição

A função δ foi implementada por meio de:

Dictionary<(string, char), (string, char, char)>

mapeando:

(estado atual, símbolo lido)

para

(novo estado, símbolo escrito, direção de movimento)

Controle de Execução

Foi implementado um contador de passos com limite configurável para evitar loops infinitos durante a execução.

Interface de Console

O conteúdo da fita é exibido com o cabeçote destacado entre colchetes, por exemplo:

1 1

facilitando o acompanhamento da execução e da transição entre estados.

4. Resultados e Testes

Nesta seção são apresentados os resultados obtidos pelos simuladores desenvolvidos.

4.1 Autômato Finito Determinístico (L1 – Cadeias terminadas em "ab")

Cadeia	Resultado Esperado	Resultado Obtido
ab	ACEITA	ACEITA
aab	ACEITA	ACEITA
bab	ACEITA	ACEITA
ababab	ACEITA	ACEITA
ba	REJEITA	REJEITA

ϵ (cadeia vazia)	REJEITA	REJEITA
b	REJEITA	REJEITA

4.2 Autômato de Pilha

Linguagem L2 = $a^n b^n$

Cadeia	Resultado Esperado	Resultado Obtido
ab	ACEITA	ACEITA
aabb	ACEITA	ACEITA
aaabbb	ACEITA	ACEITA
aab	REJEITA	REJEITA
abb	REJEITA	REJEITA
ba	REJEITA	REJEITA
ϵ (cadeia vazia)	REJEITA	REJEITA
abab	REJEITA	REJEITA

Linguagem L3 = Palíndromos

Cadeia	Resultado Esperado	Resultado Obtido
a	ACEITA	ACEITA
aba	ACEITA	ACEITA
abba	ACEITA	ACEITA
ab	REJEITA	REJEITA
aab	REJEITA	REJEITA

4.3 Máquina de Turing

Linguagem L4 = $a^n b^n c^n$

Cadeia	Resultado Esperado	Resultado Obtido
--------	--------------------	------------------

abc	ACEITA	ACEITA
aabbcc	ACEITA	ACEITA
aaabbbccc	ACEITA	ACEITA
aabbc	REJEITA	REJEITA
ab	REJEITA	REJEITA
abc abc	REJEITA	REJEITA

5. Análise Comparativa

O projeto evidencia a evolução do poder computacional ao longo da Hierarquia de Chomsky.

O AFD é eficiente para validações simples e reconhecimento de linguagens regulares, mas não possui memória auxiliar suficiente para reconhecer a linguagem $a^n b^n$.

O Autômato de Pilha supera essa limitação por meio da pilha, possibilitando o reconhecimento de linguagens livres de contexto. Entretanto, ele não consegue reconhecer adequadamente a linguagem $a^n b^n c^n$, pois necessita controlar simultaneamente três contagens dependentes.

A Máquina de Turing surge como o modelo universal de computação. Sua fita infinita e a possibilidade de múltiplas varreduras permitem reconhecer linguagens sensíveis ao contexto e realizar computações arbitrárias, conforme estabelecido pela tese de Church-Turing.

6. Conclusão

O desenvolvimento deste projeto permitiu aplicar conceitos formais da Teoria da Computação em um ambiente real de desenvolvimento de software.

A implementação em C# demonstrou que estruturas como Dictionary e HashSet são adequadas para representar funções de transição e conjuntos de estados de maneira eficiente.

Além do valor acadêmico, o estudo mostrou que autômatos possuem aplicações práticas em diversas áreas, incluindo sistemas embarcados, validação de processos, modelagem comportamental e inteligência artificial.

Conclui-se que os conceitos estabelecidos por Alan Turing, Hopcroft e Sipser continuam fundamentais para a compreensão dos limites e das possibilidades da computação moderna.

7. Referências

SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage, 2013.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.

BRANDÃO, Pedro Ramos. *Alan Turing: da necessidade do cálculo, a máquina de Turing até à computação*. Revista de Ciências da Computação, n. 12, 2017.

ARBACHE, Fernando Bastos. *AFD e a Hierarquia de Necessidades de Maslow*. 2024.

FIALHO, Arthur V. F.; NASCIMENTO, Mozart Lima. *Modelagem de um Sistema Embarcado como um AFD*. Instituto Federal da Paraíba, 2024.

POZZA, Osvaldo A.; PENEDO, Sérgio. *A Máquina de Turing: Simulação e Computabilidade*. UFSC, 2002.

NUNES, Ghabriel Calsa. *Simulador de Autômatos e Máquinas de Turing*. Trabalho de Conclusão de Curso, UFSC, 2017.